# Informe: implementación de PagoPar en Tradexpar Digital Hub

**Fecha:** 15 de abril de 2026
**Alcance:** revisión del código del repositorio `tradexpar-digital-hub` (frontend Vite/React, API Node de pagos, Supabase).

---

## 1. Resumen ejecutivo

La integración con **PagoPar** está **implementada de forma completa en el servidor de pagos Node** ( `server/` ): inicio de transacción contra la API oficial 2.0, construcción del payload (comprador + ítems), generación del token SHA1 según documentación, redirección al checkout de PagoPar, persistencia del `hash` en Supabase, **webhook** para actualizar el estado del pago y endpoints de **consulta de estado**.

El **frontend de la tienda** no llama a PagoPar directamente: crea el pedido en Supabase vía `tradexpar` , luego invoca la API externa `POST .../create-payment` y redirige al `paymentLink` devuelto. Los datos de checkout (documento, dirección, **código de ciudad hub PagoPar 1–15**) alimentan el pedido que el servidor traduce al objeto `comprador` de PagoPar.

**Nota:** `tradexpar.createPayment` en el cliente (Supabase) existe como flujo alternativo con `VITE_PAYMENT_REDIRECT_URL` ; el checkout actual usa **`api.createPayment`** (servidor de pagos con PagoPar real).

---

## 2. Arquitectura del flujo

1. **Checkout** ( `src/pages/store/CheckoutPage.tsx` ): el usuario completa datos; se elige ciudad con `pagopar_city_code` (tabla `paraguay_cities` o lista de respaldo `PAGOPAR_CIUDADES_PY` ).
2. **Pedido:** `tradexpar.createOrder` → RPC `create_checkout_order` guarda cliente y totales en `tradexpar.orders` .
3. **Pago:** `api.createPayment(order.id)` → `POST /api/public/orders/:orderId/create-payment` en el servidor Node (cabecera `x-api-key` ).
4. **Servidor:** lee el pedido desde Supabase (service role), arma payload PagoPar, POST a `iniciar-transacción` , guarda `payment_reference` , `payment_status: pending` , `pagopar_hash` ; responde con `paymentLink` = URL de checkout + hash.
5. **Navegador:** guarda `order_id` y `ref` en `sessionStorage` y hace `window.location.href = paymentLink` .
6. **Retorno / éxito:** páginas de éxito pueden consultar estado con `getPaymentStatus` / `getPaymentStatusByHash` .
7. **Webhook:** `POST /api/pagopar/webhook` (sin API key; validación por token SHA1) actualiza `payment_status` a `paid` o `failed` según cuerpo de PagoPar.

---

## 3. Backend (API de pagos Node)

**Ubicación principal:** `server/src/index.js` y módulo dedicado `server/src/pagopar.js` .

### 3.1 Endpoints relevantes

| Método y ruta | Autenticación | Función |
|---|---|---|

| | | |
|---|---|---|
| `POST /api/public/orders/:orderId/create-payment` | `x-api-key` | Inicia transacción PagoPar y devuelve enlace de pago |
| `GET /api/public/orders/:orderId/payment-status` | `x-api-key` | Estado por pedido + validación opcional de `ref` y `hash` |
| `GET /api/public/payment-status?hash=` | `x-api-key` | Resolución por `pagopar_hash` (retorno solo con hash) |
| `POST /api/pagopar/webhook` | Token PagoPar (`sha1(private_key + hash_pedido)`) | Sincroniza `payment_status` en `orders` |
| `GET /health` | No | Incluye diagnóstico de URLs PagoPar resueltas |

### 3.2 API PagoPar

- **Ruta:** `/api/comercios/2.0/iniciar-transacción` sobre la base API configurada (producción por defecto: `https://api.pagopar.com`).
- **Checkout:** `https://www.pagopar.com/pagos/{hash}` (construido en `checkoutUrlFromHash` en `pagopar.js`).
- **Token de inicio:** SHA1 de la concatenación `private_key + id_pedido_comercio + strval(floatval(monto_total))`, alineado con la documentación y con helpers `phpStrvalFloatval` / `buildStartTransactionToken`.

### 3.3 Configuración por variables de entorno

Documentadas en `server/.env.example`, entre otras:

- Credenciales: `PAGOPAR_PUBLIC_KEY`, `PAGOPAR_PRIVATE_KEY` (se desaconseja mezclar con `*_TOKEN`; el arranque valida conflictos).
- Entorno: `PAGOPAR_ENV` (`production` / `staging`), overrides `PAGOPAR_API_BASE URL`, `PAGOPAR_CHECKOUT_BASE_URL`, `PAGOPAR_STAGING_USE_OFFICIAL_HOSTS`.
- Comercio: `PAGOPAR_FORMA_PAGO`, `PAGOPAR_RETURN_URL`, `PAGOPAR_WEBHOOK_URL`, `PAGOPAR_SEND_PAYLOAD_URLS` (incluir URLs en el JSON enviado a PagoPar).
- Ítem único agregado: categoría, ciudad, id de producto, imagen, descripciones, datos de vendedor, modo `physical` / `virtual` (`PAGOPAR_PRODUCT_MODE` y overrides por body `pagopar.*`).
- Depuración: varios flags `PAGOPAR_LOG_*`, `PAGOPAR_ORDER_WRAPPER`, `PAGOPAR_SKIP_WEBHOOK_VERIFY` (solo entornos controlados).

### 3.4 Validaciones y seguridad

- Forma fija de claves en `comprador` y en cada línea de `compras_items` (aserciones antes del POST).
- Coherencia `monto_total` con la suma de `precio_total` de ítems.
- Webhook: `verifyWebhookToken` con comparación resistente a timing cuando aplica; opción de omitir verificación vía env (riesgo documentado implícitamente para dev).
- Middleware `apiKeyMiddleware` protege rutas públicas de la API de pagos excepto el webhook (diseño habitual: PagoPar no envía la API key del comercio).

### 3.5 Base de datos (Supabase / `tradexpar`)

- Columnas de pago en `orders`: `payment_reference`, `payment_status`, `pagopar_hash` (índice único parcial cuando no es nulo). Scripts: `supabase/tradexpar_orders_payment_columns.sql`.

- Datos de envío/checkout: `supabase/tradexpar_orders_checkout_shipping.sql` y comentarios alineados con PagoPar.
- Ciudades: `paraguay_cities.pagopar_city_code` — seed `tradexpar_paraguay_cities_seed.sql`; comentario de esquema en `tradexpar_paraguay_cities.sql`.

---

## 4. Frontend (Vite + React)

### 4.1 Servicio HTTP ( `src/services/api.ts` )

- `createPayment(orderId, options?)` → POST con cuerpo opcional JSON `{ pagopar: { ... } }` para overrides (modo producto, categoría, descripción, vendedor, etc.).
- `getPaymentStatus`, `getPaymentStatusByHash` para pantallas posteriores al pago.

### 4.2 Checkout ( `src/pages/store/CheckoutPage.tsx` )

- Carga ciudades con `tradexpar.listParaguayCities()`; si falla o la tabla está vacía, usa `legacyParaguayCityOptions()` desde `src/config/pagoparCiudadesPy.ts` (hubs 1–15).
- Tras `createOrder`, llama **`api.createPayment(order.id)`** sin opciones extra en el flujo por defecto (los defaults vienen del servidor por `.env`).
- Mensajería UX indica que los datos son para facturación/envío y pago con PagoPar.

### 4.3 Tipos ( `src/types/index.ts` )

- `ParaguayCity.pagopar_city_code`, `PaymentResponse` con `hash` / `pagopar_hash`, etc.

### 4.4 `tradexpar.createPayment` (Supabase)

- Actualiza `payment_reference` y `payment_status: pending` en `orders` y puede devolver un `paymentLink` desde plantilla de env; **no** es el camino usado por `CheckoutPage` para PagoPar (ese camino es la API Node).

---

## 5. Estado de madurez

| Área | Estado |
|---|---|
| Iniciar transacción + redirect | Implementado |
| Token SHA1 inicio | Implementado y alineado con fórmula documentada |
| Webhook pago/cancelación | Implementado |
| Consulta estado por pedido y por hash | Implementado |
| Catálogo ciudades / código hub | Implementado (DB + fallback TS) |
| Configuración operativa | Amplia vía `.env` (producción/staging, ítems, URLs) |
| SDK oficial PagoPar en browser | No usado (correcto: todo server-side) |

**Riesgos / mantenimiento:** URLs por defecto de retorno y webhook en el código apuntan a un host de ejemplo (Hostinger); en producción deben sustituirse por el dominio real vía variables de entorno. Los logs de depuración pueden exponer datos sensibles si se activan en producción.

---

## 6. Archivos de referencia (rutas relativas al repo)

- `server/src/index.js` — rutas create-payment, payment-status, webhook, health
- `server/src/pagopar.js` — endpoints, SHA1, `iniciarTransaccion`, `checkoutUrlFromHash`, verificación webhook
- `server/.env.example` — inventario de variables PagoPar
- `src/services/api.ts` — cliente HTTP hacia la API de pagos
- `src/pages/store/CheckoutPage.tsx` — flujo checkout → `api.createPayment`
- `src/config/pagoparCiudadesPy.ts` — códigos hub fallback
- `supabase/tradexpar_orders_payment_columns.sql`, `tradexpar_paraguay_cities*.sql` — esquema datos

---

*Documento generado a partir del análisis estático del código del repositorio.*